BIG DATA COURSE · CHAPTER

Data Warehouses

& Cloud Analytics

From Dimensional Modeling to Snowflake — Theory & Practice

Undergraduate Level · Practical Labs Included · Powered by Snowflake

Chapter Overview

What You Will Learn

01 DW Concepts

OLTP vs OLAP, star/snowflake schemas, fact & dimension tables, ETL vs ELT

02 Snowflake Architecture

Cloud-native DW design, storage/compute separation, virtual warehouses

03 Hands-on Labs

Practical exercises using Snowflake Snowsight — from loading data to query optimization

04 Review & Takeaways

Consolidate knowledge with end-of-chapter questions and key learning outcomes

01

Data Warehouse Concepts

Understanding the foundations of analytical data storage

What is a Data Warehouse?

A Data Warehouse (DW) is a centralized repository that integrates data from multiple sources to support business intelligence and analytical reporting.

Inmon's 4 Defining Characteristics

Subject-Oriented

Organized around key business subjects (sales, customers, products) rather than applications.

Integrated

Data from disparate sources is cleansed, transformed, and consolidated into a unified format.

Time-Variant

Historical data is preserved. Every record has a time element enabling trend analysis.

Non-Volatile

Data is loaded and read — not updated in place. Previous records are never deleted or modified.

OLTP vs OLAP

Two Fundamentally Different Systems

OLTP — Online Transaction Processing	
Purpose	Day-to-day operations
Users	Clerks, front-end applications
Queries	Simple (INSERT, UPDATE, DELETE)
Data	Current, real-time
Optimization	Write throughput
Example	Bank transaction, POS system

VS

OLAP — Online Analytical Processing	
Purpose	Decision support & analytics
Users	Analysts, data scientists, managers
Queries	Complex (aggregations, joins)
Data	Historical, integrated
Optimization	Read throughput
Example	Sales trend report, BI dashboard

Traditional Data Warehouse Architecture

Data flows through layers — from raw operational systems to decision-ready information.

Source Systems
ERP, CRM, Flat files, APIs

Staging Area
Raw, temporary storage

Core DW
Integrated, historical data

Data Marts
Subject-specific subsets

BI & Analytics
Reports, dashboards, ML

← ETL Process: Extract, Transform, Load →

Why Separate Layers?

Each layer serves a distinct purpose: staging handles raw ingestion, the core DW enforces data quality, marts optimize for specific teams.

Modern vs Traditional

Cloud DWs like Snowflake compress these layers using elastic compute, making the pipeline simpler and more scalable.

Star Schema

The most common dimensional modeling technique — a central Fact table surrounded by Dimension tables.

DATE DIM

[PK] date_key
year
quarter
month
day

FACT TABLE

[FK] date_key
[FK] product_key
[FK] customer_key
[FK] store_key
[M] sales_amount
[M] units_sold

PRODUCT DIM

[PK] product_key
product_name
category
brand
price

CUSTOMER DIM

[PK] customer_key
customer_name
city
segment

STORE DIM

[PK] store_key
store_name
region
manager

✓ Simple joins · Fast aggregation queries · Intuitive for business users · Denormalized for read performance

Fact & Dimension Tables

FACT TABLE

Contains:	Measurable, numeric business events
Examples:	Sales amount, quantity sold, revenue
Keys:	Foreign keys linking to dimensions
Grain:	Level of detail per row (e.g., one row per transaction)
Size:	Usually very large (millions–billions of rows)
Type:	Additive, semi-additive, non-additive

DIMENSION TABLE

Contains:	Descriptive context attributes
Examples:	Product name, customer city, date label
Keys:	Surrogate primary key (not natural key)
Size:	Relatively small (thousands–millions of rows)
Type:	Slowly Changing Dimensions (SCD) Types 1/2/3
Usage:	Filters, grouping, and labeling queries

ETL vs ELT

As cloud data warehouses grew more powerful, a major shift occurred in how data is processed.

Traditional ETL

EXTRACT
Pull from sources

TRANSFORM
Process outside DW

LOAD
Push to warehouse

⚠ Transformations happen in a separate ETL tool (e.g., Informatica, SSIS). The warehouse receives pre-processed data. Limited scalability — the ETL server is often the bottleneck.

ETL: Best for legacy systems with limited DW compute power

Popular ETL Tools: Informatica · SSIS · Talend · Fivetran

Modern ELT

EXTRACT
Pull from sources

LOAD
Raw into warehouse

TRANSFORM
Inside warehouse

✓ Raw data lands in the warehouse first. SQL-based transformations run inside the DW using its full compute power — ideal for Snowflake's elastic virtual warehouses.

ELT: Best for cloud DWs like Snowflake with massively scalable compute

Popular ELT Tools: dbt · Snowflake Tasks · Matillion

02

Snowflake Architecture

Cloud-native data warehousing — designed for scale, simplicity, and separation

Introducing Snowflake

Snowflake is a cloud-native Data Cloud platform — a fully managed data warehouse that runs on AWS, Azure, and GCP, with a unique architecture that separates storage from compute.

Fully Managed

No hardware to provision, no software to install. Snowflake handles all infrastructure, maintenance, and patching.

Multi-Cloud

Runs on AWS, Azure, and Google Cloud. Organizations avoid vendor lock-in and can choose their preferred cloud provider.

Elastic Scaling

Compute and storage scale independently and instantly. Pay only for what you use, and scale to petabytes without downtime.

Concurrent Access

Multiple virtual warehouses can query the same data simultaneously without performance degradation or contention.

Secure by Default

End-to-end encryption, role-based access control (RBAC), and MFA enforced at the platform level.

SQL + Beyond

Full ANSI SQL support plus semi-structured data (JSON, Parquet), streaming, ML functions (Cortex), and Python notebooks.

Snowflake's Three-Layer Architecture

The key innovation: storage, compute, and services are fully decoupled and scale independently.

☁ Cloud Services Layer

- Query optimization & parsing
- Infrastructure management
- Authentication & access control
- Metadata management

⚡ Query Processing Layer (Virtual Warehouses)

- Independent compute clusters (VWs)
- Multiple VWs run concurrently on same data
- Each VW has its own CPU, memory, cache
- Auto-suspend and auto-resume capability

💾 Database Storage Layer

- Columnar, compressed storage (micro-partitions)
- Fully managed by Snowflake — users don't touch files
- Stored on cloud object storage (S3 / Azure Blob / GCS)
- Supports structured + semi-structured data

💡 Unlike traditional DWs, Snowflake lets you scale compute UP or DOWN in seconds without moving or copying data

Virtual Warehouses

A Virtual Warehouse (VW) is an independent compute cluster that
storage.

VW Sizes — T-Shirt Sizing

XS	S	M	L
XL	2XL	3XL	4XL

Key Properties

Auto-Suspend: VW pauses after a period of inactivity — no charges while idle

Auto-Resume: VW starts automatically when a new query is submitted

Multi-Cluster: Scale out with multiple clusters to handle high concurrency

Isolation: Each VW processes queries independently — no contention

SQL in the Lab

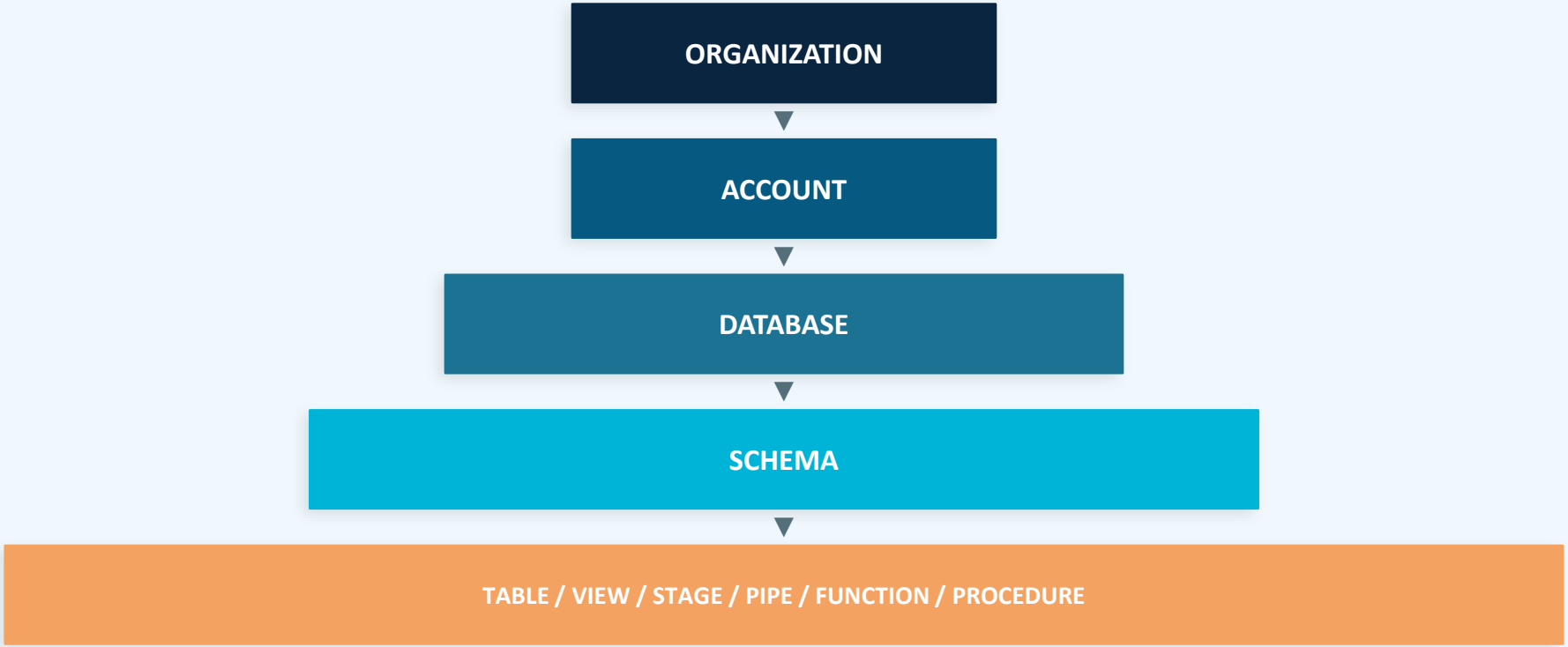
```
-- Create a virtual warehouse
CREATE WAREHOUSE my_wh
  WITH WAREHOUSE_SIZE = 'X-SMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

-- Resume / suspend manually
ALTER WAREHOUSE my_wh RESUME;
ALTER WAREHOUSE my_wh SUSPEND;

-- Resize on the fly
ALTER WAREHOUSE my_wh
  SET WAREHOUSE_SIZE = 'SMALL';
```

Snowflake Object Hierarchy

Everything in Snowflake is an object — organized in a clear hierarchy with role-based access control at each level.



Stages: Named locations (internal or external S3/Azure/GCS) where data files are stored for loading/unloading.

03

Hands-on Labs

Practical exercises using Snowflake Snowsight — learn by doing

Lab Overview

All labs use the Snowflake Academia Technical Foundations Workbook. Ensure you have downloaded and unzipped the lab files before starting.

Lab 1

Intro to Snowflake & Snowsight

- Log in & set up MFA (Passkey method)
- Explore the Snowsight UI & navigation
- Create a database, schema, and table
- Run queries — Snowbear Air airline scenario

Lab 2

Loading & Unloading Structured Data

- Upload SQL files via Workspaces
- COPY INTO command from external stage
- Load compressed (GZIP) files
- Validate data, handle load errors

Lab 3

Query Performance & Caching

- Read and interpret the Query Profile
- Understand metadata, result, data cache
- Apply partition pruning with predicates
- Use EXPLAIN to analyze execution plans

Lab 4

Semi-structured & Advanced Features

- Query JSON with dot notation syntax
- Use FLATTEN for nested arrays
- Monitor usage with ACCOUNT_USAGE schema
- Explore Cortex LLM functions (SENTIMENT, TRANSLATE)

Lab 1 — Introduction to Snowflake & Snowsight

Scenario: You are working for Snowbear Air, an airline. Your task is to create Snowflake objects in a development environment and test queries.

1. **Log In & MFA** — Access the Snowflake URL provided by your instructor. Set up MFA using the Passkey method (or Authenticator app as a fallback).
2. **Explore Snowsight** — Navigate the UI: Projects (Workspaces, Notebooks), Ingestion, AI & ML, Monitoring, and the Catalog sections.
3. **Upload SQL & Set Context** — Upload the lab .sql file to Workspaces. Set your Role, Database, Schema, and Warehouse (context) before running any SQL.
4. **Create Objects** — Run SQL to create: a Virtual Warehouse, a Database, a Schema, and a Table. Insert sample data and run SELECT queries.
5. **User Defaults** — Set your default role and warehouse in User Settings → Preferences so they are pre-selected in future sessions.

```
USE DATABASE snowbear_air_db;  USE SCHEMA dev;  SELECT * FROM flights LIMIT 10;
```

Lab 2 — Loading Data & Lab 3 — Query Performance

Lab 2 — Loading Structured Data

COPY INTO	Main command for bulk loading files from a stage into a Snowflake table
External Stage	References a cloud storage location (S3 bucket, Azure container)
File Format	Define format object: CSV, JSON, Parquet — with delimiter, encoding, etc.
GZIP Loading	Snowflake natively decompresses GZIP files during COPY — no pre-processing needed
Validation	Use COPY INTO with VALIDATION_MODE = RETURN_ERRORS to check before loading
ON_ERROR	Options: CONTINUE, SKIP_FILE, ABORT_STATEMENT — control error behavior

Lab 3 — Caching & Query Performance

Metadata Cache	Query metadata (table size, row count) without spinning up a VW — instant results
Result Cache	Re-use cached query results if the query and data haven't changed — free of charge
Data Cache	Local SSD disk cache on the VW — reduces remote storage reads for repeated scans
Query Profile	Visual execution DAG showing node-by-node performance — identify bottlenecks
Partition Pruning	WHERE clauses on clustered columns skip irrelevant micro-partitions — faster scans
EXPLAIN	Shows the query execution plan before running — useful for optimization planning

Lab 4 — Semi-structured Data & Cortex AI

Snowflake natively stores and queries semi-structured formats like JSON using the VARIANT data type.

Querying JSON Data

```
-- Dot notation: simple field
SELECT v:name::STRING
FROM my_table;

-- Nested field access
SELECT v:address:city::STRING
FROM my_table;

-- FLATTEN for arrays
SELECT f.value:item::STRING
FROM my_table,
LATERAL FLATTEN(input => v:items) f;
```

Snowflake Cortex LLM Functions

`SENTIMENT()`
Returns a score from -1 to 1 indicating positive/negative sentiment of text

`TRANSLATE()`
Translates text between languages using built-in LLM capabilities

`SUMMARIZE()`
Produces a concise summary of long text documents

`EXTRACT_ANSWER()`
Answers a question based on a given text passage

💡 Cortex functions run entirely inside Snowflake — no external APIs, no data leaving your account. This is the bridge between traditional data warehousing and modern AI-powered analytics.

04

Review & Assessment

Consolidate your understanding with these chapter questions

Chapter Review Questions

Q1

What are the four defining characteristics of a Data Warehouse according to Bill Inmon?

Hint: Think: Subject-Oriented, ?, ?, ?

Q2

A retail company needs a system to process thousands of customer orders per second. Should they use OLTP or OLAP? Explain your reasoning.

Hint: Consider: write frequency, query type, data currency

Q3

In a star schema, what is the difference between a Fact table and a Dimension table? Give one real-world example of each.

Hint: Fact = measures; Dimension = context/descriptors

Q4

Explain why ELT is generally preferred over ETL for cloud data warehouses like Snowflake.

Hint: Consider: where transformation happens and why that matters

Q5

What is a Snowflake Virtual Warehouse? How does auto-suspend save costs, and what happens when a new query is submitted to a suspended VW?

Hint: Think about the separation of storage and compute

Q6

When loading data in the lab, you encountered errors. What COPY INTO option would you use to skip files with errors and continue loading the rest?

Hint: Recall the ON_ERROR parameter options from Lab 2

Key Takeaways

1. Data Warehouses are purpose-built for analytical workloads (OLAP), not transactions (OLTP). Their design — star schemas, fact and dimension tables — optimizes for fast, complex queries over historical data.

2. ELT has replaced ETL as the standard for cloud DWs: raw data lands first, transformations run inside the warehouse using SQL, taking advantage of elastic compute.

3. Snowflake's core innovation is separating storage from compute. Virtual Warehouses scale independently and auto-suspend when idle, making it both powerful and cost-efficient.

4. Snowflake handles structured AND semi-structured data (JSON, Parquet) natively using the VARIANT type and dot notation queries — bridging the DW and data lake worlds.

5. Modern data platforms like Snowflake are evolving beyond storage/query into AI-powered analytics (Cortex LLM) — the data warehouse is now also an AI platform.